

Task 1.1 - objects and properties

Suorituksen vaatimukset

Me kaikki tiedämme What is JSON ja olemme käyttäneet sitä, mutta usein kutsumme objekteja JSONeiksi, mitä ne eivät ole, vaikka molemmat on kirjoitettu hyvin samalla tavalla. Lue lisää JSON- ja JS-objekteista: [JSON vs JavaScript Object | GeeksforGeeks](#)

Määritellään nyt, että **objekti on kokoelma ominaisuuksia**, jossa **ominaisuus on nimi-arvo-pari**. Sanan kokoelma sijaan voimme käyttää myös sanoja kuten list tai container, ne ovat melko lailla sama asia.

Toistaaksemme sen uudelleen, voimme myös sanoa, että **objekti koostuu ominaisuuksista ja ominaisuudella on nimi ja arvo**.

Ominaisuudet ovat itse asiassa muuttujia. Tämä on yleinen kaava, jota esiintyy kaikkialla tutkimusalallamme. Ajattele html-elementtien attribuutteja tai css-ominaisuuksia. Kaikilla on yhteistä sama nimi-arvo-käsite.

Luodaanpa uusi tiedostojoukko, ts. **public/shapes-demo.html** ja **src/shapes-demo.ts** ja linkitä ne yhteen edellisessä tehtävässä määritellyllä tavalla.

Muista kirjoittaa vientideklaraatio varmistaaksesi, että skripti on moduuli.

```
export {}
```

Luo .ts-tiedostoon objekti nimeltä **rectangle** jotka sisältävät seuraavat ominaisuudet, ja määritä edellisessä tehtävässä lasketut tai määritetyt arvot:

- **x** (vasemman yläkulman x-koordinaatti)
- **y** (vasemman yläkulman y-koordinaatti)
- **width**
- **height**

Tämä tehtävä on kopio edellisistä tehtävistä mutta käyttää objektia datan tallentamiseen, joten piirrämme suorakulmion käyttämällä objektissamme olevia tietoja.

Joten kun vedotaan **fillRect** funktiossa arvojen antamisen sijaan asetamme suorakulmio-objektin ominaisuudet.

Kaksi ensimmäistä parametria edustavat suorakulmion vasemman yläkulman koordinaattisijaintia, ts. **rectangle.x** ja **rectangle.y**.

Kaksi viimeistä parametria ovat suorakulmion leveys ja korkeus, ts. **rectangle.width** ja **rectangle.height**.

Task 1.2 - rehearse objects and properties

Suorituksen vaatimukset

Jatkona edelliselle tehtävälle, luo toinen objekti samaan **src/shapes-demo.ts** tiedosto nimeltä **circle** ja määritä sille seuraavalla rakenteella varustettu objekti:

- **x** (keskipisteen x-koordinaatti)
- **y** (keskipisteen y-koordinaatti)
- **radius**

Aseta **x** ja **y** keskipisteestä mihin tahansa kankaan sisällä oleviin arvoihin. Aseta myös **radius** kohtuulliseen arvoon, jotta ympyrä on kokonaan kankaan sisällä.

Käytä samaa lähestymistapaa kuin edellisessä tehtävässä ja piirrä ympyrä käyttämällä yllä kuvatulla tavalla määriteltyjä ympyräobjektin arvoja.

Näin piirrät ympyrän kankaalle: HTML Canvas Circles

Task 1.3 - methods

Suorituksen vaatimukset

Objektit voivat sisältää muutakin kuin vain ominaisuuksia. Ne voivat sisältää myös funktioita. Objektin sisällä olevaa funktiota kutsutaan metodiksi. Se on vain termi, jota käytetään objektiin kuuluvista funktioista. Ne määritellään yksinkertaisesti ominaisuuksina (eli nimi-arvo-parina), joissa arvo on funktio.

Lukea JavaScript Object Methods

Tärkein etu on, että pelkkien datan/ominaisuuksien objekteiksi jäsentämisen lisäksi määrittelemme samassa objektissa myös toiminnot, jotka voidaan suorittaa kyseisille datalle/ominaisuuksille.

Autolla on ominaisuuksia, kuten esim.

- `color`
- `enginePower`
- `fuelType`

, mutta sillä on myös toimintoja, eli

- `start()`
- `drive()`
- `break()`

Kaikki nämä ominaisuudet ja funktiot/metodit kuuluvat samaan objektiin, joten on luonnollista, että ne ovat yhdessä osana samaa objektia.

Tehtäväsi on lisätä metodi suorakulmio-objektiin nimeltä `draw` ja siirrä suorakulmion piirtävää koodia funktion sisällä. Sen sijaan, että suorakulmioon pääsisit käyttämällä `rectangle` muuttuja, käytä sitä käyttämällä `this` avainsana, kuten yllä olevassa linkissä on esitetty.

Lue lisää aiheesta JavaScript `this`

Muistaa: `this` viittaa nykyiseen objektiin vain, kun sitä käytetään objektiin kuuluvan funktion määritelmän sisällä.

Seuraavaksi suorita `draw()` suorakulmion metodi, joka piirtää suorakulmion.

Tee sama ympyräobjektille.

Side task 1.a - value vs reference

Suorituksen vaatimukset

Muuttujalla on arvo. Arvo voi olla alkeellista (primitive) tyyppiä, kuten **number**, **string**, **boolean**.

Otetaan esimerkiksi seuraava koodi:

```
const a = 7
let b = a
b = 8
console.log(a, b) // prints 7 8
```

Arvon muuttaminen **b** ei muuta arvoa kohdassa **a**.

Tämä on esimerkki muuttujista, jotka sisältävät arvoja, ja joissa vain yhden muuttujan arvoa voi muuttaa.

Toisaalta **object** (mukaan lukien **array** tyyppi) ei ole muuttujan sisällä, vaan muuttuja sisältää viittauksen (reference) objektiin (se tietää, missä objekti on tallennettu, se tietää sen osoitteen muistissa).

Voit huomata tämän kokeilemalla seuraavaa esimerkkiä:

```
const c = { course: "TS" }
const d = c
d.course = "JS"
console.log(c, d) // prints {course: 'JS'} {course: 'JS'}
```

Jos kuitenkin annamme d-muuttujalle kokonais uuden arvon, edellinen objekti ei tietenkään muutu.

```
const c = { course: "TS" }
let d = c
d = { course: "JS" }
console.log(c, d) // prints {course: 'TS'} {course: 'JS'}
```

Harjoittele kaikkia näitä tiedostossa, esim. sidetasks/val-ref.js (voit suorittaa sen komennolla `node js-tiedoston suorittamiseksi` komennolla `node` päätteessä **node file-name**). Samat periaatteet pätevät TS:ään.

Tee kaikissa kolmessa edellä mainitussa tapauksessa myös seuraavat toimenpiteet:

- Luo oliolle metodi, joka muuttaa olion ominaisuutta.
- Luo global funktio parametrilla, jossa parametri on liitetty määriteltyyn objektiin.
- funktion sisältä muuttaa objektin ominaisuutta tai kutsua objektin metodia, joka muuttaa ominaisuutta.
- Ensimmäisessä ja kolmannessa tapauksessa annat funktion parametrille uuden arvon/objektin.

Tämä auttaa sinua ymmärtämään paremmin, mitä **passed by value** ja **passed by reference** tarkoittaa.

Task 1.4 - objects with similar properties

Suorituksen vaatimukset

Palataanpa tarinaamme eli muotoihin (shapes). Piirretään lisää suorakulmioita samalla lähestymistavalla kuin edellisessä muototehtävässä. Työskennelkää saman tiedoston parissa, nykyinen koodi on edelleen pätevä eikä muutu, lisäämme vain lisää koodia.

Tällä hetkellä meillä on yksi suorakulmiomuuttuja, joka sisältää viittauksen objektiin. Luodaan kaksi suorakulmiota lisää ja piirretään ne. Meidän on määriteltävä kaksi muuta objektia samalla rakenteella, jotka sisältävät `x`, `y`, `width`, `height` ominaisuudet ja `draw()` menetelmä, eli funktio.

Sitten vedota `draw()` metodi jokaiselle objektille, joka piirtää suorakulmiot.